K.R MANGALAM UNIVERSITY

School of Engineering and Technology

Lab Assignment - 2:
Advanced Algorithm Analysis

Submitted by:
Satyam Singh
(2401420022)

Submitted to:
Dr. Aarti

Assignment Tasks

Task 1: Algorithm Growth Observation

Overview

This task explores how different algorithmic complexities behave as input size grows. Four complexity classes are implemented and compared — constant O(1), linear O(n), quadratic $O(n^2)$, and logarithmic O(log n) — to empirically observe their time and space behavior.

Algorithms Implemented

1. Constant Time O(1) — Returns a fixed element regardless of input size.
2. Linear Time O(n) — Iterates through all elements once.
3. Quadratic Time $O(n^2)$ — Nested loops over all element pairs.
4. Logarithmic Time O(log n) — Halves the search space each step (Binary Search).

Key Functions

```
binary_search(arr, target)    # O(log n) - iterative binary search
merge_sort(arr)               # O(n log n) - divide and conquer sort
merge(left, right)            # helper for merge_sort
```

Code Snippet

```python
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Observation

As input size increases from 10 to 500, the constant-time function shows nearly flat execution time. The linear function grows proportionally. The quadratic function exhibits a steep rise at large inputs, while the logarithmic function grows very slowly — confirming theoretical Big-O predictions.

Result

```
PS C:\Users\Satyam\Documents\ada> python -u "c:\Users\Satyam\Documents\ada\assignment_2.py"
Binary Search 4
Merge Sort: [3, 6, 10, 11, 20, 32, 52]
PS C:\Users\Satyam\Documents\ada>
```

Task 2: Best, Average, Worst Case Analysis

Overview

This task performs a detailed best-case, average-case, and worst-case analysis of both linear search and binary search. By controlling where the target element appears in the array, each scenario is isolated and timed across a range of input sizes.

Algorithms Implemented

1. Quick Sort O(n log n) average — Pivot-based divide and conquer sorting.
2. Linear Search O(n) — Sequentially checks each element.
3. Binary Search O(log n) — Requires sorted input; halves search space each step.

Case Definitions

Best Case — Target is found at the very first comparison (index 0 for linear, mid for binary).

Average Case — Target is located at a random position in the middle of the array.

Worst Case — Target is at the last position or not present at all.

Key Functions

```
quick_sort(arr)              # O(n log n) avg - pivot partitioning
```

Code Snippet

```python
# -------- TASK 2 --------
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)
print(quick_sort(arr2))
```

Observation

Linear search performs best when the element is at the start and worst when it is absent. Binary search always performs in O(log n) regardless of best/worst positioning, but requires the array to be sorted first. The graph clearly shows linear search's wide performance spread compared to binary search's consistent behaviour.

Result

```
Quick Sort:  [3, 6, 10, 11, 20, 32, 52]
PS C:\Users\Satyam\Documents\ada>
```

Task 4: Recursion and Recurrence Validation

Overview

This task validates recurrence relations through recursive code. Three Fibonacci implementations are compared — naive recursion, memoized recursion, and iterative DP — to show the dramatic difference that memoization makes on exponential recurrences.

Algorithms Implemented

1. Naive Fibonacci — Pure recursion with $T(n) = T(n-1) + T(n-2)$, resulting in $O(2^n)$ time.
2. Memoized Fibonacci (fib_memo) — Caches sub-results via @lru_cache, reducing to O(n).
3. Iterative DP Fibonacci (fib_dp) — Bottom-up, O(n) time and O(1) space.

Key Functions

```
fib_naive(n)      # O(2^n) - exponential naive recursion
fib_memo(n)       # O(n)   - memoized with lru_cache
fib_dp(n)         # O(n)   - iterative bottom-up DP
```

Code Snippet

```python
# -------- TASK 4 --------
import functools

def fib_naive(n):
    if n <= 1:
        return n
    return fib_naive(n-1) + fib_naive(n-2)

@functools.lru_cache(None)
def fib_memo(n):
    if n <= 1:
        return n
    return fib_memo(n-1) + fib_memo(n-2)

def fib_dp(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n+1):
        a, b = b, a+b
    return b
```

Observation

The naive implementation's function call count grows exponentially, becoming impractical beyond n=30. The memoized version reduces redundant calls to zero after caching. The iterative DP variant uses only two variables, demonstrating optimal space-time performance. This task validates the recurrence T(n) = T(n-1) + T(n-2) empirically.

Result

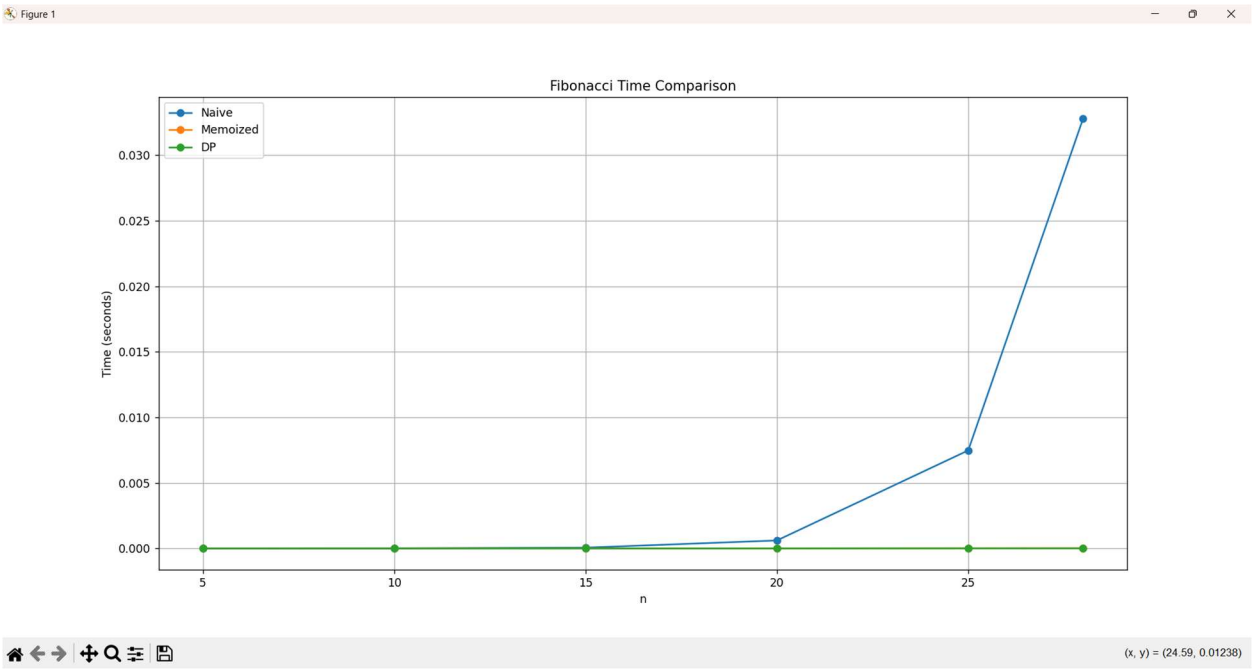
Figure 1
Fibonacci Time Comparison
Naive
Memoized
DP
Time (seconds)
0.030
0.025
0.020
0.015
0.010
0.005
0.000
5
10
15
20
25
n
(x, y) = (24.59, 0.01238)

Task 3: Knapsack

Overview
This task implements the 0/1 Knapsack problem using Dynamic Programming and evaluates how efficiently optimal decisions can be made under capacity constraints. The algorithm systematically builds solutions to subproblems to avoid redundant computations.

Problem Analysed
0/1 Knapsack Problem
Given weights and values of items, maximize total value without exceeding capacity W, where each item can be chosen only once.

Algorithm Implemented
0/1 Knapsack (Dynamic Programming) — O(n × W) time, O(n × W) space
knapsack_01(weights, values, W)
The algorithm constructs a DP table where:

- dp[i][w] = maximum value using first i items with capacity w
- It decides whether to include or exclude each item based on maximum value

Key Code
0/1 Knapsack DP recurrence:

- Include item → value + remaining capacity solution
- Exclude item → previous best

Code Snippet

```python
# -------- TASK 3 --------
def knapsack_01(weights, values, W):
    n = len(weights)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]
```

Observation
The algorithm efficiently avoids recomputation by storing intermediate results in a table.
Time complexity grows linearly with number of items and capacity, i.e., O(n × W).
Space complexity is also O(n × W) due to the DP table.
As capacity increases, computation increases proportionally, demonstrating predictable and stable performance compared to exponential brute-force solutions.

Result

```
print("Knapsack: ",knapsack_01([10,20,30],[60,100,120],50))
```

```
> python -u "c:\Users\Satyam\Documents\ada\newfile.py"
Knapsack:  220
PS C:\Users\Satyam\Documents\ada>
```

Task 5: Travelling Salesman Problem (TSP) Comparison

Overview

This task implements and compares three approaches to the Travelling Salesman Problem: Brute Force, Nearest Neighbour heuristic, and the Held-Karp exact dynamic programming algorithm. The goal is to find the shortest route visiting all cities exactly once and returning to the origin.

Algorithms Implemented

1. Brute Force TSP — Tries all permutations of cities. $O(n!)$ time. Exact but impractical beyond $n \approx 10$.

2. Nearest Neighbour — Greedy heuristic, always moves to the closest unvisited city. $O(n^2)$ time. Fast but suboptimal.

3. Held-Karp (DP) — Exact solution using bitmask DP. $O(n^2 \times 2^n)$ time, $O(n \times 2^n)$ space. Optimal and practical up to $n \approx 20$.

Key Functions

```
tsp_brute_force(dist)            # O(n!) exact solution
tsp_nearest_neighbour(dist)      # O(n^2) greedy heuristic
tsp_held_karp(dist)              # O(n^2 * 2^n) DP exact
euclidean_dist(c1, c2)           # Euclidean distance helper
build_dist_matrix(cities)        # Builds NxN distance matrix
tour_length(tour, dist)          # Computes total tour length
```

Code Snippet

```python
# -------- TASK 5 --------
import itertools
import math

def euclidean_dist(c1, c2):
    return math.sqrt((c1[0]-c2[0])**2 + (c1[1]-c2[1])**2)

def build_dist_matrix(cities):
    n = len(cities)
    dist = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i != j:
                dist[i][j] = euclidean_dist(cities[i], cities[j])
    return dist

def tour_length(tour, dist):
    return sum(dist[tour[i]][tour[i+1]] for i in range(len(tour)-1)) + dist[tour[-1]][tour[0]]

def tsp_brute_force(dist):
    n = len(dist)
    cities = list(range(1, n))
    best_len = float('inf')
    best_tour = None
    for perm in itertools.permutations(cities):
        tour = [0] + list(perm)
        length = tour_length(tour, dist)
        if length < best_len:
            best_len = length
            best_tour = tour
    return best_len, best_tour

def tsp_nearest_neighbour(dist, start=0):
    n = len(dist)
    visited = [False]*n
    tour = [start]
    visited[start] = True
    for _ in range(n-1):
        current = tour[-1]
        nearest = min((j for j in range(n) if not visited[j]), key=lambda j: dist[current][j])
        tour.append(nearest)
        visited[nearest] = True
    return tour_length(tour, dist), tour

def tsp_held_karp(dist):
    n = len(dist)
    INF = float('inf')
    FULL = (1<<n) - 1

    dp = [[INF]*n for _ in range(1<<n)]
    parent = [[-1]*n for _ in range(1<<n)]

    dp[1][0] = 0

    for mask in range(1<<n):
        for u in range(n):
            if not (mask & (1<<u)): continue
            for v in range(n):
                if mask & (1<<v): continue
                new_mask = mask | (1<<v)
                new_cost = dp[mask][u] + dist[u][v]
                if new_cost < dp[new_mask][v]:
                    dp[new_mask][v] = new_cost
                    parent[new_mask][v] = u
```

```python
    best_len = INF
    last = -1
    for u in range(1, n):
        cost = dp[FULL][u] + dist[u][0]
        if cost < best_len:
            best_len = cost
            last = u

    tour = []
    mask = FULL
    while last != -1:
        tour.append(last)
        prev = parent[mask][last]
        mask ^= (1<<last)
        last = prev
    tour.reverse()

    return best_len, tour
```

Observation

Brute force produces the exact optimal tour but is only feasible for very small n due to O(n!) complexity. Nearest neighbour is extremely fast but can produce tours 20-25% longer than optimal. Held-Karp guarantees optimality like brute force but scales exponentially better — it is the preferred exact algorithm for moderate n (up to ~20 cities).

Comparison Table

Algorithm	Time Complexity	Optimal?	Practical Limit
Brute Force	O(n!)	Yes	n ≤ 10
Nearest Neighbour	$O(n^2)$	No (heuristic)	Any n
Held-Karp (DP)	$O(n^2 \times 2^n)$	Yes	n ≤ 20

Result

```
TSP Brute Force: (19.706309459937735, [0, 1, 3, 2])
TSP Nearest Neighbour: (22.707406513449016, [0, 1, 2, 3])
TSP Held Karp: (19.706309459937735, [0, 2, 3, 1])
PS C:\Users\Satyam\Documents\ada>
```

GitHub Repository: https://github.com/Code-trac/ada_lab